

ĐỒ ÁN CAPSTONE 2

FALLGUARD AI

Hệ thống phát hiện té ngã thời gian thực
dựa trên ước lượng tư thế và mô hình chuỗi thời gian

TÀI LIỆU MÔ TẢ TOÀN BỘ DỰ ÁN

Dữ liệu → tiền xử lý → AI → xác minh sự kiện → webcam/web → cảnh
báo Telegram

Sinh viên: _____

Mã số sinh viên: _____

Giảng viên hướng dẫn: _____

Đơn vị: _____

Tóm tắt

FallGuard AI là một hệ thống phát hiện té ngã end-to-end từ camera, được xây dựng cho đồ án Capstone 2. Hệ thống không phân loại độc lập từng ảnh. Thay vào đó, nó trích xuất 33 điểm khớp cơ thể bằng MediaPipe Pose, chuẩn hóa theo tâm hông và chiều dài thân, tạo chuỗi đặc trưng chuyển động, sau đó dùng mô hình tuần tự nhân quả GRU để dự đoán đồng thời tư thế và xác suất sự kiện té ngã. Một state machine tiếp tục kiểm tra tư thế nằm, thời gian bất động, sự hồi phục và thời gian cooldown trước khi phát cảnh báo. Kiến trúc này giúp tách hai câu hỏi: “khung hình hiện tại trông giống tư thế nào?” và “chuỗi chuyển động vừa xảy ra có phải là một sự kiện té ngã hay không?”.

Dự án hiện có pipeline tải và xử lý UR Fall Detection Dataset (URFD), huấn luyện GRU/LSTM/TCN, hiệu chỉnh ngưỡng chỉ trên validation, đánh giá theo frame và theo sự kiện, suy luận trên video/webcam, FastAPI, dashboard web, SQLite, webhook và cảnh báo Telegram kèm ảnh. Bản thực nghiệm hiện tại dùng toàn bộ 70 sequence camera 0 của URFD ở profile preview, tạo 7.960 frame sau resample, 795 cửa sổ train, 121 cửa sổ validation và 156 cửa sổ test. Checkpoint GRU tốt nhất có 153.316 tham số, dừng sớm tại epoch 16 và chọn epoch 9. Trên test độc lập, event recall đạt 1,000, event precision đạt 0,600, event F1 đạt 0,750 và số cảnh báo giả là 0,222/video.

Phạm vi sử dụng

Đây là prototype nghiên cứu và demo phần mềm. Nó không phải thiết bị y tế, không được dùng làm cơ chế gọi cấp cứu duy nhất, và không thay thế đánh giá của người chăm sóc hoặc nhân viên y tế.

Contents

Tóm tắt	i
1 Giới thiệu bài toán	1
1.1 Bối cảnh	1
1.2 Mục tiêu của đề án	1
1.3 Đóng góp kỹ thuật chính	2
2 Kiến trúc tổng thể	3
2.1 Luồng end-to-end	3
2.2 Hai pipeline: offline và online	3
2.3 Cấu trúc repository	3
3 Nguồn dữ liệu và cách tạo nhãn	5
3.1 UR Fall Detection Dataset	5
3.2 Hai profile tải dữ liệu	5
3.3 Ý nghĩa nhãn posture	6
3.4 Dẫn xuất nhãn fall-event	6
3.5 Chia tập và chống rò rỉ dữ liệu	6
4 Tiền xử lý pose và đặc trưng	8
4.1 Trích xuất 33 landmarks	8
4.2 Xử lý landmark bị mất	8
4.3 Chuẩn hóa tâm hông và tỷ lệ thân	9
4.4 Cấu tạo vector 239 chiều	9
4.5 Cửa sổ nhân quả	10
5 Mô hình AI	11

5.1	Vì sao chọn mô hình chuỗi thời gian	11
5.2	Kiến trúc đa nhiệm	11
5.3	Hàm mất mát và mất cân bằng lớp	12
5.4	Thông số checkpoint thật	12
6	Huấn luyện, calibration và đánh giá	13
6.1	Quy trình huấn luyện	13
6.2	Hiệu chỉnh ngưỡng không chạm test	13
6.3	Metric theo frame và theo sự kiện	14
6.4	Kết quả test độc lập	14
7	State machine xác minh té ngã	16
7.1	Tại sao cần state machine	16
7.2	EMA xác suất	16
7.3	Các trạng thái	16
7.4	Đo bất động	17
7.5	Cấu hình MVP và cấu hình demo	17
8	Suy luận video và webcam	18
8.1	Video inference	18
8.2	Webcam realtime	18
8.3	Ba chế độ demo	19
9	Backend, dashboard và lưu trữ	20
9.1	FastAPI	20
9.2	SQLite event store	20
9.3	Dashboard	21
10	Cảnh báo Telegram và webhook	22
10.1	Luồng Telegram	22
10.2	Bảo mật token	22
10.3	Webhook	23
10.4	Quyền riêng tư	23
11	Hướng dẫn chạy dự án từ đầu	24

11.1	Yêu cầu	24
11.2	Bước 1 - Cài môi trường	24
11.3	Bước 2 - Smoke test không cần dataset công khai	24
11.4	Bước 3 - Tải URFD	25
11.5	Bước 4 - Trích pose và tạo window	25
11.6	Bước 5 - Train	25
11.7	Bước 6 - Calibration	25
11.8	Bước 7 - Đánh giá test	26
11.9	Bước 8 - Video inference	26
11.10	Bước 9 - Webcam realtime	26
11.11	Bước 10 - Dashboard web	27
11.12	Bước 11 - Telegram	27
11.13	Bước 12 - Test và lint	28
12	Kịch bản demo bảo vệ đồ án	29
12.1	Demo web ổn định	29
12.2	Demo realtime	29
12.3	Thông điệp trình bày	29
13	Hạn chế, rủi ro và hướng phát triển	31
13.1	Hạn chế dữ liệu và mô hình	31
13.2	Hạn chế đánh giá	31
13.3	An toàn và đạo đức	31
13.4	Lộ trình phát triển	32
14	Kết luận	33
A	Bảng tham số cấu hình	34
B	Xử lý lỗi thường gặp	35
C	Câu hỏi bảo vệ gợi ý	36

List of Figures

2.1	Kiến trúc xử lý end-to-end của FallGuard AI.	3
5.1	Mô hình GRU đa nhiệm được dùng cho checkpoint hiện tại.	11
6.1	Các metric chính của checkpoint sau calibration.	15
7.1	Luồng trạng thái xác minh sự kiện.	16
8.1	Snapshot annotated được trích tại thời điểm xác nhận fall trong video demo. . .	19

List of Tables

2.1	So sánh pipeline huấn luyện và pipeline vận hành.	3
3.1	Thông tin dữ liệu URFD được dùng trong dự án.	5
3.2	Mapping nhãn URFD sang nhãn nội bộ.	6
3.3	Phân bố sequence theo split trong manifest hiện tại.	7
4.1	Các nhóm đặc trưng trên mỗi frame.	9
5.1	Thông số checkpoint GRU tốt nhất của thí nghiệm.	12
6.1	Kết quả thực tế trên 156 test windows thuộc 9 sequence có window hợp lệ. . .	14
7.1	Khác biệt giữa profile thực tế và profile benchmark.	17
8.1	Mục đích của các giao diện demo.	19
9.1	Các endpoint chính.	20
A.1	Các tham số chính của cấu hình MVP.	34
B.1	Troubleshooting nhanh.	35

Giới thiệu bài toán

1.1 Bối cảnh

Té ngã là một sự kiện diễn ra nhanh nhưng hậu quả có thể kéo dài. Một hệ thống camera giám sát không chỉ cần nhận ra hình ảnh một người đang nằm, vì người dùng có thể chủ động nằm xuống, ngồi thấp, cúi nhặt đồ hoặc tập thể dục. Bài toán đúng là nhận ra **chuỗi biến đổi tư thế theo thời gian**, sau đó xác minh tình trạng sau ngã trước khi gửi cảnh báo. Vì vậy FallGuard dùng ba lớp quyết định:

1. **Pose estimation**: biến pixel thành các điểm khớp cơ thể, giảm ảnh hưởng của màu quần áo và nền.
2. **Mô hình chuỗi thời gian**: học cách tư thế và vận tốc thay đổi trong khoảng hai giây gần nhất.
3. **State machine**: áp dụng điều kiện thời gian, bất động, hồi phục và chống gửi trùng để tạo sự kiện có thể giải thích.

1.2 Mục tiêu của đồ án

- Xây dựng pipeline có thể tái lập từ tải dữ liệu đến checkpoint và báo cáo đánh giá.
- Phát hiện té ngã theo thời gian thực từ một camera RGB thông thường.
- Tạo kết quả có thể giải thích: tư thế, xác suất, trạng thái, bằng chứng, ảnh chụp và lịch sử sự kiện.
- Hỗ trợ ba cách demo: video có sẵn, webcam trực tiếp và dashboard web.
- Gửi thông báo Telegram kèm ảnh khi sự kiện chuyển sang `CONFIRMED_FALL`.
- Đánh giá trung thực theo sequence, tránh rò rỉ frame giữa train và test.

1.3 Đóng góp kỹ thuật chính

Điểm khác biệt so với một ứng dụng CRUD

Đóng góp cốt lõi nằm ở pipeline pose-sequence nhân quả, mô hình đa nhiệm, cơ chế xác minh sau ngã và đánh giá theo sự kiện. Dashboard, SQLite và Telegram là lớp ứng dụng giúp trình diễn và vận hành kết quả AI, không phải phần thay thế cho nghiên cứu mô hình.

Các đóng góp có thể trình bày trong buổi bảo vệ gồm:

- Chuỗi 40 frame chỉ dùng quá khứ và frame hiện tại, không nhìn tương lai.
- Hai đầu ra dùng chung encoder: phân loại tư thế và xác suất fall-event.
- Nhãn fall-event được dẫn xuất có provenance rõ ràng thay vì đánh đồng “nằm” với “té ngã”.
- Threshold được chọn trên validation, sau đó giữ nguyên khi đánh giá test.
- State machine kết hợp xác suất, tư thế nằm, bất động và hồi phục.
- Toàn bộ luồng camera - AI - sự kiện - SQLite - ảnh - Telegram hoạt động cục bộ và có test tự động.

Kiến trúc tổng thể

2.1 Luồng end-to-end



Figure 2.1: Kiến trúc xử lý end-to-end của FallGuard AI.

2.2 Hai pipeline: offline và online

Table 2.1: So sánh pipeline huấn luyện và pipeline vận hành.

Thành phần	Offline: tạo mô hình	Online: sử dụng mô hình
Đầu vào Pose	URFD, annotation, video/ảnh Trích toàn bộ sequence và lưu NPZ	Webcam hoặc file video tải lên Trích từng frame ngay khi nhận được
Đặc trưng	Tạo và lưu các window train/validation/test	Buffer giữ 40 frame mới nhất
AI	Train, early stopping, calibration, evaluation	Nạp <code>best.pt</code> , chạy forward
Sự kiện	Metrics theo frame và event	State machine xác minh, lưu SQLite
Đầu ra	Checkpoint, history, metrics JSON	Video annotated, event JSON, ảnh, Telegram

2.3 Cấu trúc repository

```

src/fallguard/
|-- data/          # URFD download, annotation, manifest, synthetic
|-- pose/          # MediaPipe Pose va tao NPZ

```

```
|-- features/      # noi suy, chuan hoa, geometry, causal windows
|-- models/       # GRU, LSTM, TCN, rule baseline, predictor
|-- training/     # dataset loader, loss, train, metrics, calibration
|-- inference/    # buffer, video/webcam, state machine, snapshot
|-- api/          # FastAPI, schema, SQLite, webhook
|-- notifications/ # Telegram sendMessage/sendPhoto
|-- web/          # dashboard HTML/CSS/JavaScript
`-- cli.py        # tat ca lenh fallguard
```

Các thư mục dữ liệu tuân theo nguyên tắc tách lớp:

- data/raw: dữ liệu tải nguyên bản;
- data/interim: skeleton/pose trung gian;
- data/processed: cửa sổ số đã sẵn sàng cho PyTorch;
- data/manifests: danh sách sequence và split;
- artifacts: checkpoint, metadata, metric, calibration;
- runtime: upload tạm, video kết quả, snapshot và SQLite;
- demo_videos: chín video dùng để trình diễn.

Nguồn dữ liệu và cách tạo nhãn

3.1 UR Fall Detection Dataset

Nguồn dữ liệu chính là UR Fall Detection Dataset (URFD) do Kwolek và Kępski công bố [2, 1]. Trang chính thức mô tả 70 sequence, gồm 30 sequence té ngã mô phỏng và 40 hoạt động sinh hoạt hằng ngày (Activities of Daily Living - ADL). Fall sequence có hai camera Kinect; ADL chỉ có camera 0. Do đó dự án chọn camera 0 để mọi sequence có chung góc nhìn.

Table 3.1: Thông tin dữ liệu URFD được dùng trong dự án.

Thuộc tính	Giá trị
Số sequence	70: 30 fall và 40 ADL
Camera dùng	Camera 0
Profile mặc định	Preview MP4, depth và RGB ghép ngang
Phần ảnh đưa vào pose	Nửa RGB bên phải của preview
FPS gốc / FPS đích	Khoảng 30 FPS / 20 FPS
Giấy phép	CC BY-NC-SA 4.0, mục đích học thuật phi thương mại
Nguồn annotation	Depth-feature CSV chính thức của URFD

3.2 Hai profile tải dữ liệu

Downloader hỗ trợ:

preview

Các MP4 composite độ phân giải nhỏ, phù hợp để tải lập nhanh trên máy cá nhân. Tổng 70 video camera 0 khoảng 97 MB trong bản làm việc.

rgb Các ZIP ảnh PNG RGB camera 0 đầy đủ, dung lượng nén khoảng 4,49 GB, chất lượng tốt hơn nhưng tải và xử lý lâu hơn.

smoke

Chỉ tải một fall và một ADL để kiểm tra kết nối/pipeline.

Downloader có resume bằng file `.part`, retry, kiểm tra Content-Length, CRC của ZIP, ngăn path traversal và symlink khi giải nén. Vì máy chủ URFD không công bố checksum chuẩn cho mọi file, dự án không tự tạo ra checksum giả rồi tuyên bố là checksum nguồn.

3.3 Ý nghĩa nhãn posture

URFD cung cấp giá trị posture theo frame:

Table 3.2: Mapping nhãn URFD sang nhãn nội bộ.

Nhãn URFD	Nhãn FallGuard	Ý nghĩa
-1	UPRIGHT (0)	Không nằm
0	TRANSITION (1)	Chuyển tiếp trong lúc ngã
1	LYING (2)	Nằm trên mặt đất

Đây là nhãn **tư thế**, không phải nhãn sự kiện. Một ADL cũng có thể chứa chuyển tiếp hoặc chủ động nằm xuống. Nếu ánh xạ trực tiếp LYING $\Rightarrow$ FALL, hệ thống sẽ học sai bản chất bài toán.

3.4 Dẫn xuất nhãn fall-event

FallGuard tạo một target nhị phân riêng:

- Với sequence fall: event dương bắt đầu từ vùng transition bền vững và kéo qua impact/đoạn nằm ngắn sau impact.
- Với sequence ADL: toàn bộ event target là 0, kể cả khi người tham gia chủ động nằm xuống.
- Provenance được ghi là `urfd_depth_feature_derived`, để chỉ rõ đây là nhãn dẫn xuất từ annotation chứ không phải nhãn tay độc lập.

Quyết định này cho phép đầu posture học hình dạng cơ thể, còn đầu event học diễn biến của một lần té. Đây là lý do kiến trúc dùng multi-task thay vì một classifier “fall/not fall” trên từng ảnh.

3.5 Chia tập và chống rò rỉ dữ liệu

Toàn bộ frame/window của một sequence được giữ trong cùng một split. Không chia ngẫu nhiên từng frame, vì các frame liên kề gần như giống nhau và sẽ làm metric test cao giả tạo. Manifest

hiện tại có:

Table 3.3: Phân bố sequence theo split trong manifest hiện tại.

Split	Sequence	Fall	ADL	Frame gốc trong manifest
Train	49	21	28	8.703
Validation	11	5	6	1.518
Test	10	4	6	1.715
Tổng	70	30	40	11.936

URFD không cung cấp mapping subject đủ đáng tin cậy cho toàn bộ sequence. Vì vậy kết quả được gọi chính xác là **cross-sequence**, không tuyên bố là cross-subject.

Tiền xử lý pose và đặc trưng

4.1 Trích xuất 33 landmarks

MediaPipe Pose Landmarker phát hiện 33 vị trí cơ thể và trả về tọa độ cùng độ tin cậy [3]. Mỗi frame được lưu dưới dạng:

$$\mathbf{P}_t \in \mathbb{R}^{33 \times 4}, \quad \mathbf{p}_{t,j} = (x_{t,j}, y_{t,j}, z_{t,j}, v_{t,j}),$$

trong đó v là visibility. Mask `valid[T, 33]` cho biết landmark có dùng được hay không. Pose NPZ có contract:

```
landmarks          float32 [T, 33, 4]
valid              bool    [T, 33]
timestamps_ms      int64   [T]
frame_indices      int64   [T]
posture_labels     int64   [T]
event_labels       int64   [T]
annotation_provenance scalar string
```

Pipeline resample video về 20 FPS theo timestamp, không giả định camera luôn giữ FPS tuyệt đối. Đối với preview composite, nửa RGB bên phải được crop trước khi đưa vào MediaPipe. Quá trình chuẩn bị có checkpoint `.partial`; nếu gián đoạn, chương trình tiếp tục từ frame đã hoàn thành.

4.2 Xử lý landmark bị mất

Một landmark có thể mất do che khuất, ánh sáng hoặc người ra khỏi khung hình. FallGuard chỉ nội suy tuyến tính gap ngắn không quá ba frame và chỉ khi hai đầu gap đều có quan sát hợp lệ:

$$\hat{\mathbf{p}}_t = (1 - \alpha)\mathbf{p}_{t_0} + \alpha\mathbf{p}_{t_1}, \quad \alpha = \frac{t - t_0}{t_1 - t_0}.$$

Ba mask được giữ riêng: quan sát thật, điểm có thể sử dụng và điểm được nội suy. Gap dài không bị bìa dữ liệu; vị trí không hợp lệ được đưa về 0 và validity vẫn được bảo toàn.

4.3 Chuẩn hóa tâm hông và tỷ lệ thân

Tâm hông là trung bình hai hông:

$$\mathbf{c}_t = \frac{\mathbf{p}_{t,Lhip} + \mathbf{p}_{t,Rhip}}{2}.$$

Tâm vai được tính tương tự. Scale là khoảng cách Euclid giữa tâm vai và tâm hông:

$$s_t = \|\mathbf{s}_t - \mathbf{c}_t\|_2, \quad \tilde{\mathbf{p}}_{t,j} = \frac{\mathbf{p}_{t,j} - \mathbf{c}_t}{s_t}.$$

Chuẩn hóa này làm giảm ảnh hưởng của vị trí người trong ảnh và khoảng cách đến camera. Nếu chỉ thấy một hông, pipeline dùng median half-hip vector từ các frame hợp lệ; nếu scale hiện tại không tính được, dùng median scale của sequence.

4.4 Cấu tạo vector 239 chiều

Mỗi frame được biểu diễn bởi:

Table 4.1: Các nhóm đặc trưng trên mỗi frame.

Nhóm	Công thức số chiều	Số chiều
Tọa độ normalized x, y, z	33×3	99
Visibility	33×1	33
Vận tốc landmark	33×3	99
Hình học cơ thể	8 đặc trưng	8
Tổng	$99 + 33 + 99 + 8$	239

Vận tốc rời rạc được tính bằng:

$$\mathbf{v}_{t,j} = \tilde{\mathbf{p}}_{t,j} - \tilde{\mathbf{p}}_{t-1,j}.$$

Nếu landmark không hợp lệ ở một trong hai frame, velocity tương ứng bằng 0. Tám đặc trưng hình học gồm góc thân so với phương thẳng đứng, chiều dài thân, chiều rộng vai, chiều

rộng hông, góc gối trái/phải, tọa độ y tương đối của mũi và trung bình y của hai cổ chân.

4.5 Cửa sổ nhân quả

Mỗi mẫu là 40 frame ở 20 FPS, tương đương khoảng hai giây. Stride bằng 5, tức khoảng 0,25 giây tạo một dự đoán mới. Với cửa sổ $[t - 39, t]$, target chính là nhãn của frame cuối t :

$$\mathbf{X}_t = [\mathbf{x}_{t-39}, \dots, \mathbf{x}_t] \in \mathbb{R}^{40 \times 239}.$$

Không có feature hoặc label sau t được đọc. Đây là contract nhân quả, bảo đảm logic train giống logic webcam realtime.

Mô hình AI

5.1 Vì sao chọn mô hình chuỗi thời gian

Một frame người nằm không đủ kết luận ngã. GRU ghi nhớ biến đổi từ đứng sang chuyển tiếp rồi nằm, đồng thời nhẹ hơn nhiều kiến trúc video 3D-CNN. Repository cũng hỗ trợ LSTM và TCN qua model registry để làm ablation, nhưng checkpoint hiện tại dùng GRU.

5.2 Kiến trúc đa nhiệm

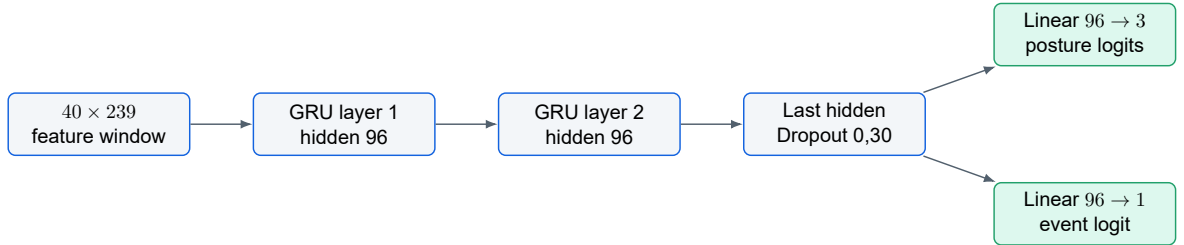


Figure 5.1: Mô hình GRU đa nhiệm được dùng cho checkpoint hiện tại.

Mô hình là GRU một chiều, hai tầng, hidden size 96, dropout 0,30. Hidden state cuối cùng đi qua hai linear head:

$$\mathbf{h}_t = \text{GRU}(\mathbf{X}_t), \quad (5.1)$$

$$\mathbf{z}_t^{\text{posture}} = W_p \mathbf{h}_t + b_p, \quad (5.2)$$

$$z_t^{\text{event}} = W_e \mathbf{h}_t + b_e, \quad (5.3)$$

$$\hat{p}_t^{\text{event}} = \sigma(z_t^{\text{event}}). \quad (5.4)$$

Posture head có ba logit cho UPRIGHT, TRANSITION, LYING; event head có một logit nhị phân. Hai task chia sẻ encoder, nên thông tin tự thể đóng vai trò regularization và giải thích cho

dự đoán event.

5.3 Hàm mất mát và mất cân bằng lớp

Hàm mất mát tổng:

$$\mathcal{L} = \mathcal{L}_{event} + \lambda_p \mathcal{L}_{posture}, \quad \lambda_p = 0,30.$$

$\mathcal{L}_{posture}$ là weighted cross entropy; $\mathcal{L}_{event}$ là binary cross entropy with logits với positive weight. Trên tập train hiện tại:

- posture counts: 443 upright, 124 transition, 228 lying;
- event counts: 714 negative, 81 positive;
- posture weights: 0,4604; 1,6449; 0,8946;
- event positive weight: $714/81 \approx 8,8148$.

Việc đặt trọng số làm giảm xu hướng mô hình luôn đoán lớp phổ biến. AdamW dùng learning rate 10^{-3} , weight decay 10^{-4} , batch size 32. Seed bằng 42 và chế độ deterministic được lưu trong metadata.

5.4 Thông số checkpoint thật

Table 5.1: Thông số checkpoint GRU tốt nhất của thí nghiệm.

Thuộc tính	Giá trị
Architecture	Causal GRU
Input size	239 feature/frame
Window	40 frame @ 20 FPS
Hidden size / layers	96 / 2
Dropout	0,30
Số tham số	153.316, đều trainable
Thiết bị train	CUDA
Best epoch	9
Epoch đã chạy	16/30, dừng sớm
Best validation loss	0,35755
Train / validation samples	795 / 121

Huấn luyện, calibration và đánh giá

6.1 Quy trình huấn luyện

Mỗi epoch chạy train với optimizer và validation không cập nhật gradient. Sau mỗi epoch, hệ thống lưu `last.pt`; nếu validation loss tốt hơn, lưu `best.pt`. Nếu không cải thiện trong bảy epoch liên tiếp, early stopping dừng quá trình. Artifact gồm:

- `best.pt`, `last.pt`: trọng số, optimizer, config;
- `config.yaml`: cấu hình snapshot của experiment;
- `metadata.json`: model spec, class weights, seed, provenance;
- `history.csv`: loss/accuracy từng epoch;
- `metrics.json`: metric validation;
- `calibration.json`: threshold và đường cong threshold;
- `test_metrics_calibrated.json`: kết quả test cuối.

6.2 Hiệu chỉnh ngưỡng không chạm test

Threshold mặc định 0,5 không nhất thiết tối ưu cho event. Hệ thống quét 101 giá trị trên validation và chọn threshold theo event-level F1; tie-break lần lượt là recall, precision và threshold nhỏ hơn. API calibration từ chối đường dẫn hoặc metadata có nhãn `test`. Kết quả chọn $\tau = 0,86$; threshold này được giữ nguyên cho test.

6.3 Metric theo frame và theo sự kiện

Frame metrics xem từng cửa sổ là một mẫu. Event-level metrics gom các vùng dương liên tục theo sequence, sau đó ghép một-một vùng dự đoán và vùng ground truth nếu chúng giao nhau. Các công thức chính:

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F_1 = \frac{2PR}{P + R}.$$

False alarms/video bằng tổng false-positive event chia số video. Detection delay là chênh lệch giữa thời điểm bắt đầu vùng dự đoán và vùng thật cho các event ghép thành công.

6.4 Kết quả test độc lập

Table 6.1: Kết quả thực tế trên 156 test windows thuộc 9 sequence có window hợp lệ.

Metric	Giá trị
Posture accuracy	0,840
Posture macro F1	0,615
Frame event precision	0,429
Frame event recall	1,000
Frame event F1	0,600
Event-level precision	0,600
Event-level recall	1,000
Event-level F1	0,750
True / false / missed events	3 / 2 / 0
False alarms/video	0,222

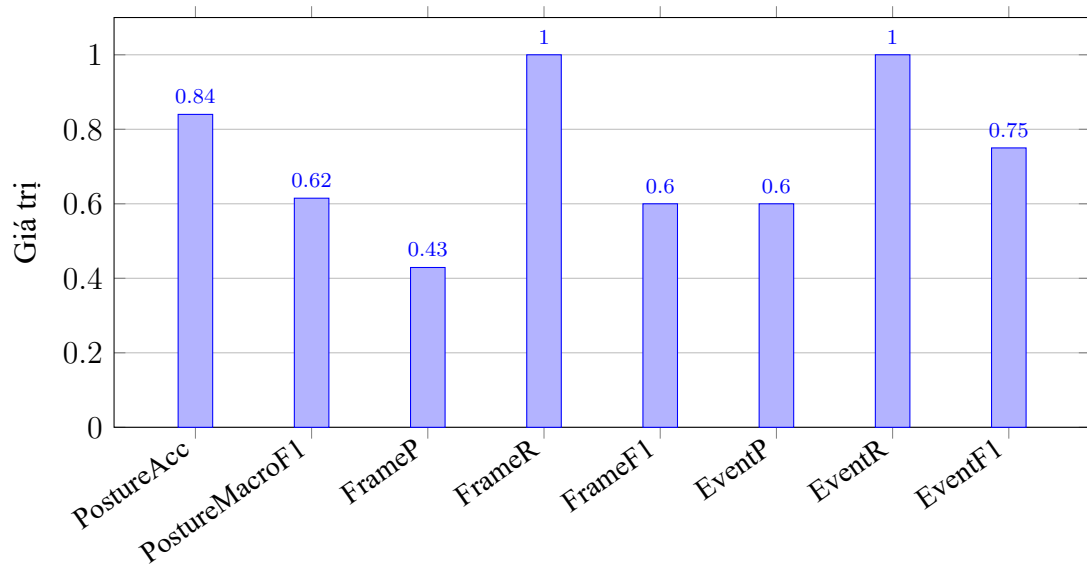


Figure 6.1: Các metric chính của checkpoint sau calibration.

Cách diễn giải đúng

Recall 1,0 trên test nhỏ nghĩa là không bỏ sót ba ground-truth event có thể đánh giá trong split này; nó không chứng minh hệ thống không bỏ sót ngoài đời thực. Precision 0,6 và 0,222 false alarm/video cho thấy vẫn có cảnh báo giả. Lớp TRANSITION chỉ có 7 mẫu test và F1 bằng 0,10, là điểm yếu rõ ràng cần báo cáo trung thực.

State machine xác minh té ngã

7.1 Tại sao cần state machine

Neural network cho xác suất ở từng thời điểm nhưng cảnh báo là quyết định có ngưỡng cảnh. State machine làm mượt xác suất, kiểm tra thời lượng và tạo một event ID duy nhất. Nó giảm cảnh báo do spike ngắn và ngăn gửi Telegram lặp lại trên nhiều frame CONFIRMED_FALL.

7.2 EMA xác suất

Xác suất được làm mượt:

$$\bar{p}_t = \alpha p_t + (1 - \alpha) \bar{p}_{t-1}, \quad \alpha = 0,40.$$

Ngưỡng candidate trong cấu hình MVP là 0,70; ngưỡng high-confidence là 0,82. Profile demo dùng ngưỡng xác nhận 0,86 để khớp threshold calibration.

7.3 Các trạng thái

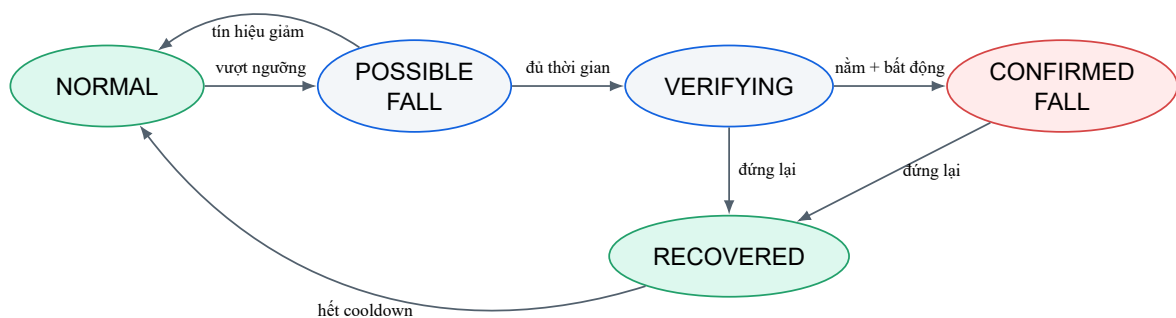


Figure 7.1: Luồng trạng thái xác minh sự kiện.

NORMAL

Không có candidate; risk thấp.

POSSIBLE_FALL

EMA vượt ngưỡng candidate; bắt đầu đo thời gian.

VERIFYING

Tín hiệu tồn tại đủ lâu; theo dõi nằm, motion và hồi phục.

CONFIRMED_FALL

Đã thấy high-confidence, nằm đủ lâu và bất động đủ lâu; risk cao, tạo event ID, lưu ảnh và gửi cảnh báo.

RECOVERED

Người đứng lại đủ lâu; ghi nhận self-recovery và chờ hết cooldown trước khi trở về normal.

7.4 Đo bất động

Với hai pose liên tiếp, hệ thống lấy median khoảng cách Euclid của các joint cùng hợp lệ. Nếu motion không vượt 0,012 thì bắt đầu/tích lũy bất động. Nếu mất pose, có ít hơn ba joint chung, hoặc gap lớn hơn một giây, bộ đếm bất động bị reset. Cách làm bảo thủ này tránh xác nhận do camera không nhìn thấy người.

7.5 Cấu hình MVP và cấu hình demo

Table 7.1: Khác biệt giữa profile thực tế và profile benchmark.

Tham số	mvp.yaml	urfd_demo.yaml
Possible threshold	0,70	0,70
Confirm threshold	0,82	0,86
Candidate duration	0,25 s	0,10 s
Lying duration	2,00 s	0,50 s
Inactive duration	8,00 s	0,00 s
Recovery upright	1,50 s	1,00 s
Cooldown	20,00 s	5,00 s

URFD clip chỉ dài vài giây, nên profile demo bỏ yêu cầu bất động tám giây. Khi demo webcam trong điều kiện thực tế, nên dùng `configs/mvp.yaml` hoặc tinh chỉnh có kiểm chứng, không dùng shortcut benchmark để tuyên bố an toàn.

Suy luận video và webcam

8.1 Video inference

Video được đọc bằng OpenCV, resample về target FPS, chạy pose và đưa frame hợp lệ/place-holder vào sequence buffer. Khi đủ 40 frame và tới stride, model tạo prediction. Kết quả được vẽ skeleton, state, posture và fall probability lên video; event được xuất JSON.

Luồng web lưu upload tạm dưới runtime/uploads, chạy inference trong threadpool, tạo video annotated và JSON dưới runtime/results, rồi xóa upload gốc. Khi vừa xác nhận ngã, hệ thống trích đúng frame theo timestamp từ video annotated và lưu JPEG tại runtime/results/snapshots.

8.2 Webcam realtime

Webcam dùng cùng feature/model/state machine với video. Network Telegram chạy trong một worker nền duy nhất để không chặn vòng lặp camera. Hệ thống gửi theo cờ “vừa xác nhận” trong metadata, không phải chỉ kiểm tra state; nhờ đó một event chỉ gửi một lần dù trạng thái xác nhận còn tồn tại qua nhiều frame.



Figure 8.1: Snapshot annotated được trích tại thời điểm xác nhận fall trong video demo.

8.3 Ba chế độ demo

Table 8.1: Mục đích của các giao diện demo.

Chế độ	Nguồn vào	Mục đích
CLI video	File cục bộ	Debug pipeline và xuất annotated video/JSON
CLI camera	Webcam/USB camera	Trình diễn phát hiện trực tiếp trên máy tính
Dashboard web	File upload vào local-host	Giao diện để dùng, lịch sử, metrics, feedback
Skeleton API	Tọa độ 33 khớp	Tích hợp từ hệ thống pose bên ngoài; Telegram text-only

Backend, dashboard và lưu trữ

9.1 FastAPI

FastAPI cung cấp validation theo type hint và OpenAPI/Swagger UI tự động [4]. Ứng dụng chạy mặc định tại <http://127.0.0.1:8000>; đây là localhost, không tự tải video lên Internet.

Table 9.1: Các endpoint chính.

Method	Endpoint	Chức năng
GET	/health	Trạng thái dịch vụ, database và model
GET	/v1/model	Metadata model, window, FPS và output
POST	/v1/predict/skeleton	Dự đoán từ chuỗi 33 landmarks
POST	/v1/predict/video	Upload, phân tích video, tạo artifact
GET	/v1/events	Danh sách sự kiện có filter/pagination
GET	/v1/events/{event_id}	Chi tiết một event
POST	/v1/events/{event_id}/feedback	Gán nhãn review đúng/sai
GET	/v1/metrics/summary	Tổng hợp số event/feedback

Upload chỉ nhận các phần mở rộng video cho phép, giới hạn mặc định 250 MB và thời lượng xử lý tối đa năm phút. Tên file server dùng UUID thay vì tên do người dùng cung cấp để tránh ghi đè/path traversal.

9.2 SQLite event store

Mỗi event có các trường: event ID, state, risk, timestamp, camera ID, person ID, source, fall probability, inactive seconds, self-recovery, evidence, metadata, feedback và thời gian tạo/cập nhật. UPSERT cho phép một event chuyển từ confirmed sang recovered mà không tạo bản ghi trùng.

Feedback gồm label, ghi chú, reviewer và thời gian. Đây là nền tảng để thu thập false alarm có kiểm duyệt và tái huấn luyện về sau.

9.3 Dashboard

Dashboard HTML/CSS/JavaScript không cần build frontend riêng. Người dùng thả video, nhập Camera ID/Person ID, chọn cờ URFD preview, nhấn phân tích và xem:

- trạng thái cuối, risk, xác suất và timeline;
- video annotated và JSON kết quả;
- lịch sử sự kiện từ SQLite;
- số liệu tổng hợp và feedback review.

Camera ID chỉ là nhãn nguồn, ví dụ CAM-DEMO-01, không phải index webcam. Person ID có thể để trống nhưng hữu ích khi lọc lịch sử.

Cảnh báo Telegram và webhook

10.1 Luồng Telegram

Khi state machine tạo `just_confirmed=True`:

1. event ID được thêm vào set chống trùng trước khi scheduling;
2. snapshot annotated được lưu cục bộ;
3. event đã lưu SQLite (có `created_at`) được format thành tiếng Việt;
4. ảnh JPEG và caption được gửi qua phương thức `sendPhoto` của Telegram Bot API; nếu ảnh lỗi, hệ thống chuyển sang `sendMessage`;
5. timeout/lỗi mạng được trả thành kết quả có cấu trúc, không làm inference dừng.

Telegram Bot API là HTTP interface và yêu cầu `chat_id` là định danh cuộc trò chuyện đích [5]. Bot username hoặc link `t.me/...` không phải Chat ID cá nhân. Người nhận phải mở bot và gửi `/start` trước khi bot có thể gửi tin.

10.2 Bảo mật token

Token được đọc từ biến môi trường hoặc file `.env` cục bộ. File này nằm trong `.gitignore`; biến đã có trong PowerShell được ưu tiên hơn `.env`. Token không được đưa vào CLI argument, repr, log hoặc error message. Bất kỳ token nào đã dán vào chat/Git/ảnh chụp phải được revoke và tạo lại bằng BotFather.

Các biến hỗ trợ:

```
FALLGUARD_TELEGRAM_BOT_TOKEN=  
FALLGUARD_TELEGRAM_CHAT_ID=  
FALLGUARD_TELEGRAM_CHAT_IDS=
```

```
FALLGUARD_TELEGRAM_ENABLED=true  
FALLGUARD_TELEGRAM_TIMEOUT_SECONDS=8
```

10.3 Webhook

Ngoài Telegram, hệ thống có thể POST event JSON đến dịch vụ khác. URL lấy từ `FALLGUARD_ALERT_WEBHOOK` hoặc `FALLGUARD_WEBHOOK_URL`; secret tùy chọn dùng cho xác thực. Webhook và Telegram dùng cùng logic dedupe nhưng là hai adapter độc lập.

10.4 Quyền riêng tư

Web dashboard chạy localhost, nhưng ảnh gửi Telegram sẽ rời máy tính và đi vào hạ tầng Telegram. Chỉ bật tính năng khi người xuất hiện trong video đồng ý; giới hạn Chat ID; bảo vệ cả snapshot cục bộ; có lịch xóa dữ liệu; không đặt camera ở khu vực riêng tư.

Hướng dẫn chạy dự án từ đầu

11.1 Yêu cầu

- Windows 10/11, PowerShell và kết nối Internet cho bước tải dữ liệu/model;
- webcam nếu demo realtime;
- GPU CUDA là tùy chọn, CPU vẫn chạy được;
- đủ dung lượng: preview nhẹ hơn nhiều so với profile RGB đầy đủ.

11.2 Bước 1 - Cài môi trường

Tại thư mục gốc repository:

```
Set-ExecutionPolicy -Scope Process Bypass
.\scripts\bootstrap.ps1
.\.venv\Scripts\Activate.ps1
fallguard doctor
```

Bootstrap tải uv và CPython 3.12 vào workspace, tạo `.venv`, đọc `uv.lock` và cài các dependency. `fallguard doctor` kiểm tra cấu hình, package, accelerator và storage.

11.3 Bước 2 - Smoke test không cần dataset công khai

```
fallguard demo --epochs 3
```

Lệnh tạo skeleton synthetic, materialize window, train vài epoch và evaluate. Nếu bước này chạy được, toolchain PyTorch/pipeline cơ bản đã hoạt động.

11.4 Bước 3 - Tải URFD

Đọc giấy phép trước, sau đó:

```
fallguard data download-urfd `
--selection full `
--profile preview `
--accept-license `
--no-extract
```

Muốn kiểm tra nhanh, thay full bằng smoke. Không nên xóa file .part khi download gián đoạn vì downloader có thể resume.

11.5 Bước 4 - Trích pose và tạo window

```
fallguard data prepare-urfd --source preview
fallguard data validate
```

Kết quả quan trọng:

```
data/interim/urfd_preview/poses/*.npz
data/manifests/urfd_preview_sequences.csv
data/processed/urfd_preview/windows_train.npz
data/processed/urfd_preview/windows_validation.npz
data/processed/urfd_preview/windows_test.npz
```

11.6 Bước 5 - Train

```
fallguard train `
--data-dir data/processed/urfd_preview `
--output artifacts/urfd_preview_gru
```

Theo dõi history.csv. Không chọn model bằng test result; checkpoint tốt nhất chỉ dựa trên validation loss.

11.7 Bước 6 - Calibration

```
fallguard calibrate `
  artifacts/urfd_preview_gru/best.pt `
  --validation-data data/processed/urfd_preview `
  --output artifacts/urfd_preview_gru/calibration.json
```

11.8 Bước 7 - Đánh giá test

```
fallguard evaluate `
  artifacts/urfd_preview_gru/best.pt `
  --data-dir data/processed/urfd_preview `
  --split test `
  --calibration artifacts/urfd_preview_gru/calibration.json `
  --output artifacts/urfd_preview_gru/test_metrics_calibrated.json
```

11.9 Bước 8 - Video inference

Video bình thường:

```
fallguard infer video my-video.mp4 `
  --checkpoint artifacts/urfd_preview_gru/best.pt
```

Video preview URFD phải bật crop:

```
fallguard infer video demo_videos/01_FALL_CONFIRMED_URFD_fall-19.mp4 `
  --checkpoint artifacts/urfd_preview_gru/best.pt `
  --config configs/urfd_demo.yaml `
  --urfd-preview
```

11.10 Bước 9 - Webcam realtime

```
fallguard infer camera `
  --source 0 `
  --checkpoint artifacts/urfd_preview_gru/best.pt `
  --config configs/mvp.yaml `
  --camera-id CAM-DEMO-01 `
  --person-id USER-001 `
```

```
--telegram
```

Nhấn Q hoặc Esc để dừng. Nếu máy có nhiều camera, thử `--source 1`. Không dùng một người lớn tuổi hoặc người có bệnh nền để mô phỏng ngã; demo an toàn trên nệm và có người hỗ trợ.

11.11 Bước 10 - Dashboard web

Khởi động nhanh đúng profile demo:

```
powershell -ExecutionPolicy Bypass `
-File .\demo_videos\START_WEB_DEMO.ps1
```

Hoặc chạy trực tiếp:

```
fallguard serve `
--checkpoint artifacts/urfd_preview_gru/best.pt `
--config configs/urfd_demo.yaml `
--open
```

Mở các địa chỉ sau trong trình duyệt:

- dashboard: <http://127.0.0.1:8000>;
- tài liệu Swagger: <http://127.0.0.1:8000/docs>.

Với file có tên chứa URFD, dashboard tự bật crop preview; video điện thoại/webcam bình thường không bật tùy chọn này.

11.12 Bước 11 - Telegram

```
powershell -ExecutionPolicy Bypass `
-File .\scripts\configure_telegram.ps1
```

Script yêu cầu token mới nhập ẩn, hướng dẫn /start, discovery Chat ID, lưu .env và gửi ảnh mẫu. Có thể kiểm tra lại:

```
fallguard telegram test `
--snapshot .\demo_videos\telegram_samples\01_FALL_CONFIRMED_snapshot.jpg
```

Nếu cấu hình khi server đang chạy, phải dừng bằng Ctrl+C và khởi động lại để process nạp .env mới.

11.13 Bước 12 - Test và lint

```
pytest -q  
ruff check .
```

Tại thời điểm hoàn thiện Telegram/dotenv, toàn bộ suite có 77 test pass. Test bao phủ downloader, parser, split, preprocessing, model shape, training smoke, calibration, state machine, video, API, SQLite, evidence và Telegram mock.

Kịch bản demo bảo vệ đồ án

12.1 Demo web ổn định

1. Chạy `START_WEB_DEMO.ps1`.
2. Chọn `01_FALL_CONFIRMED_URFD_fall-19.mp4`.
3. Xác nhận cờ “Video mẫu URFD” đang bật.
4. Nhập `CAM-DEMO-01`, nhấn phân tích.
5. Trình bày video annotated, state `CONFIRMED_FALL`, probability, lịch sử SQLite và tin Telegram.
6. Chọn một ADL normal để chứng minh không phải video nào cũng cảnh báo.

Chín mẫu gồm hai confirmed fall, các ADL normal/hard negative và các case `VERIFYING`. Case biên rất hữu ích để giải thích hệ thống biết từ chối khi chưa đủ bằng chứng.

12.2 Demo realtime

Đặt webcam sao cho thấy toàn thân, camera cố định, đủ sáng và nền không quá rối. Chờ buffer đủ hai giây. Người demo có thể mô phỏng chuyển từ đứng sang ngồi/nằm trên nệm, nhưng phải ưu tiên an toàn. Với profile MVP, cần nằm và bất động đủ thời gian; vì vậy demo realtime dài hơn video benchmark.

12.3 Thông điệp trình bày

Một flow thuyết trình ngắn:

1. Pixel được đổi thành skeleton 33 khớp để giảm phụ thuộc ngoại hình.

2. 40 frame tạo lịch sử hai giây, GRU học động học chứ không nhìn một ảnh.
3. Hai head phân biệt posture và event.
4. State machine xác minh nằm/bất động và chống cảnh báo giả.
5. Event được lưu, có snapshot và gửi Telegram để người chăm sóc kiểm tra.

Hạn chế, rủi ro và hướng phát triển

13.1 Hạn chế dữ liệu và mô hình

- URFD nhỏ, fall được diễn, không đại diện đầy đủ người cao tuổi thực tế.
- Split hiện tại cross-sequence, chưa chứng minh tổng quát cross-subject.
- Transition ít mẫu và metric thấp.
- Một camera RGB khó xử lý che khuất, thiếu sáng, góc nhìn lạ hoặc camera rung.
- Pipeline không có multi-person tracking; người nổi bật nhất có thể thay đổi.
- Nằm chủ động, quỳ, tập thể dục, ngồi nhanh hoặc xe lăn có thể tạo false alarm.

13.2 Hạn chế đánh giá

Test chỉ có chín sequence tạo được window đánh giá và ba ground-truth event ở metric event-level. Khoảng tin cậy chưa được báo cáo. Detection delay 0 trong artifact frame-level không bao gồm toàn bộ thời gian hậu kiểm 8 giây của profile MVP. Do đó không nên dùng metric này để tuyên bố cảnh báo y tế tức thì.

13.3 An toàn và đạo đức

- Có consent rõ ràng; ưu tiên lưu skeleton thay video gốc khi có thể.
- Mã hóa Person ID, hạn chế quyền xem SQLite/snapshot và đặt retention.
- Không gửi ảnh cho Chat ID ngoài nhóm chăm sóc được ủy quyền.
- Không tự động gọi cấp cứu chỉ dựa trên một model prototype.
- Cho phép người kiểm duyệt feedback false alarm/missed event.

13.4 Lộ trình phát triển

1. Thu tập dữ liệu webcam tự quay với consent: đứng, đi, ngồi, cúi, nằm chủ động, chuyển tư thế nhanh, ánh sáng/góc nhìn khác nhau.
2. Split theo người khi có subject ID; đánh giá domain gap URFD → webcam.
3. Ablation: tọa độ; +velocity; +geometry; có/không state machine.
4. So sánh GRU, LSTM, TCN trên cùng split và compute budget.
5. Quét inactivity 2/4/6/8 giây để đo trade-off false alarm và latency.
6. Thêm person tracking và hỗ trợ nhiều người.
7. Quantization/ONNX để giảm latency CPU; benchmark FPS, P50/P95 và RAM.
8. Dashboard realtime qua WebSocket, phân quyền người dùng và audit log.
9. Escalation: Telegram lần một, caregiver xác nhận, rồi mới gọi dịch vụ khác.

Kết luận

FallGuard AI là một prototype hoàn chỉnh từ dữ liệu đến ứng dụng. Thiết kế chủ đạo là biến video thành pose, biến pose thành đặc trưng thời gian, dùng GRU đa nhiệm để dự đoán posture và fall-event, rồi dùng state machine để chuyển dự đoán thành sự kiện có thể giải thích. Hệ thống đã có checkpoint và kết quả đánh giá thật, chạy được trên video/webcam, cung cấp API/dashboard, lưu SQLite và gửi Telegram kèm ảnh.

Điểm quan trọng khi trình bày không phải là khẳng định hệ thống đã đủ dùng trong y tế, mà là chứng minh một quy trình kỹ thuật đúng: dữ liệu có nguồn và giấy phép; nhân có provenance; split tránh leakage; threshold không tối ưu trên test; runtime nhân quả; cảnh báo có hậu kiểm; metric và giới hạn được báo cáo trung thực. Đây là nền móng tốt để mở rộng thành nghiên cứu cross-subject, đa dữ liệu và thử nghiệm thực địa trong giai đoạn tiếp theo.

Bảng tham số cấu hình

Table A.1: Các tham số chính của cấu hình MVP.

Khóa	Giá trị	Ý nghĩa
project.seed	42	Seed tái lập
project.device	auto	CUDA nếu có, nếu không CPU
data.target_fps	20	Tần số xử lý
data.window_size	40	Số frame trong một window
data.stride	5	Khoảng cách hai window
data.max_interpolation_ gap	3	Gap landmark tối đa được nội suy
data.min_pose_visibility	0,35	Ngưỡng landmark hợp lệ
features.normalize_ center	hip	Tâm chuẩn hóa
features.normalize_scale	torso	Scale theo thân
model.architecture	gru	Encoder chính
model.hidden_size	96	Hidden dimension
model.num_layers	2	Số recurrent layer
model.dropout	0,30	Regularization
training.batch_size	32	Batch train
training.epochs	30	Epoch tối đa
training.learning_rate	0,001	Learning rate AdamW
training.patience	7	Early stopping patience
event_detection.ema_ alpha	0,40	Mức làm mượt probability
event_detection. inactive_seconds	8,0	Thời gian bất động cần xác minh
api.max_upload_mb	250	Kích thước upload tối đa

Xử lý lỗi thường gặp

Table B.1: Troubleshooting nhanh.

Hiện tượng	Nguyên nhân và cách xử lý
Không mở được camera	Thử <code>--source 1</code> ; đóng Zoom/Teams; kiểm tra quyền camera.
Camera chạy nhưng không predict	Chờ đủ 40 frame; bảo đảm thấy toàn thân và đủ sáng.
Video URFD hiển thị depth+RGB	Bật <code>--urfd-preview</code> hoặc checkbox tương ứng.
Video thường bị crop mất nửa	Tắt URFD preview cho video điện thoại/webcam.
Telegram 400 chat not found	Chat ID sai; gửi <code>/start</code> , chạy discover và dùng ID số chính xác.
Telegram 401/404	Token sai, đã revoke hoặc process đang giữ token cũ; tạo token mới và restart server.
Telegram 409 getUpdates	Bot đang có webhook/poller khác; dừng process hoặc xóa webhook có chủ đích.
Không có ảnh Telegram	Skeleton API không có pixel hoặc snapshot lỗi; hệ thống fallback text.
Port 8000 bận	Dừng server cũ bằng <code>Ctrl+C</code> hoặc chọn port khác.
CUDA không dùng được	Chạy <code>fallguard doctor</code> ; hệ thống có thể fallback CPU.

Câu hỏi bảo vệ gợi ý

Tại sao không dùng YOLO để nhận diện người nằm?

Một bounding box hoặc class trên một frame không thể hiện quá trình chuyển từ đứng sang ngã. Pose + temporal model mô tả động học; state machine xác minh sau ngã. YOLO vẫn có thể bổ sung cho person detection/tracking nhưng không thay thế toàn bộ bài toán thời gian.

Tại sao dùng GRU?

GRU phù hợp chuỗi ngắn, ít tham số hơn LSTM, chạy causal và đủ nhẹ cho webcam. Repository vẫn có LSTM/TCN để so sánh khoa học.

Tại sao cần hai head?

Posture là nhãn có sẵn và giải thích trạng thái hình học; event là câu hỏi theo thời gian. Multi-task giúp encoder học cả tư thế lẫn diễn biến.

Tại sao recall cao nhưng precision thấp?

Class fall hiếm và mục tiêu ưu tiên không bỏ sót làm model nhạy. State machine, calibration và dữ liệu hard-negative được dùng để giảm false alarm, nhưng test vẫn cho thấy hai false-positive event.

Web có gửi video lên Internet không?

Không khi chạy ở 127.0.0.1; browser gửi file cho FastAPI trên cùng máy. Telegram là ngoại lệ: snapshot được tải lên Telegram khi bật cảnh báo.

Metric hiện tại có đủ triển khai thực tế không?

Không. Cần dữ liệu người thật có consent, split theo người, kiểm thử nhiều góc camera, confidence interval, latency và false alarms/hour trước khi cân nhắc pilot thực địa.

Bibliography

- [1] Bogdan Kwolek và Michal Kępski, “Human fall detection on embedded platform using depth maps and wireless accelerometer,” *Computer Methods and Programs in Biomedicine*, 117(3), 489–501, 2014. DOI: <https://doi.org/10.1016/j.cmpb.2014.09.005>.
- [2] University of Rzeszów, “UR Fall Detection Dataset,” <https://fenix.ur.edu.pl/~mkepski/ds/uf.html>, truy cập tháng 09/2026.
- [3] Google AI Edge, “Pose landmark detection guide,” https://developers.google.com/edge/mediapipe/solutions/vision/pose_landmarker, truy cập tháng 09/2026.
- [4] FastAPI, “Tutorial - User Guide,” <https://fastapi.tiangolo.com/tutorial/>, truy cập tháng 09/2026.
- [5] Telegram, “Telegram Bot API,” <https://core.telegram.org/bots/api>, truy cập tháng 09/2026.